

# 把任何文字变成一篇精美文章： 一套能被复用的生产流程

让结果稳定，而不是偶尔出彩

把长文生产从“模型碰巧写得好”变成一条可复现的流程：输入先统一成一份事实来源，写作交给受主题约束的组件协议，关键决策停在检查点由人拍板，状态写进文件，质量交给三路独立审核，修复限制在最小切片。

一次基于视频证据的工程拆解



《Harness 实践：将任何文字编辑成精美的文章》 · code 秘密花园 · 2026-06-17 ·

20 分 37 秒

平台字幕不可用；faster-whisper base (CPU int8) 逐行审校：606 段中 3 段标记为不确定

<https://www.bilibili.com/video/BV1ayLD6uERL>

## 目录

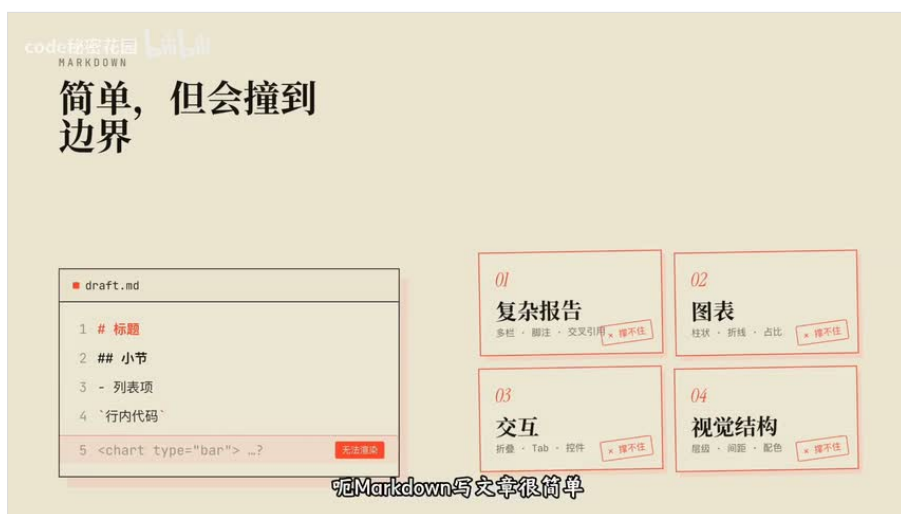
|                             |   |
|-----------------------------|---|
| 1 长文交给 Agent，卡住的往往不是模型写得不够好 | 1 |
| 2 真正可复用的，是这套工作方法            | 2 |
| 3 把“让模型写 HTML”换成“让模型组合组件”   | 3 |
| 4 整套流程实际跑起来，是九个步骤和三个检查点     | 5 |
| 5 把它压成一张可以逐条自查的清单           | 8 |

## 1 长文交给 Agent，卡住的往往不是模型写得不够好

---

你手上有一份长材料，想让它变成一篇能直接发出去的文章：有排版、有配色、有节奏感，不用再找一位设计师。第一反应通常是走 Markdown，因为写起来足够简单。可材料一旦包含复杂的报告、图表和交互结构，Markdown 就不够用了。<sup>1</sup>

HTML 看起来是更合适的载体。它的信息密度高，表格、SVG、看板、片段和公式可以随意组合；标题层级、颜色和间距能比较精准地控制；折叠、Tab 这类可交互控件也做得出来；分享出去只要一个链接，对方不需要额外安装任何东西。<sup>2</sup>



**图 1** 画面把两种载体并排放在一起：左侧是 Markdown 的极简语法——标题、小节、列表、引用、行内代码；右侧是 HTML 的四种能力——复杂报告、图表、交互、视觉结构。这张图就是“简单，但会撞到边界”的对照。<sup>3</sup>

换到 HTML 也有代价，代价出现在让模型直接手写 HTML 和 CSS 的时候：内容一长、视觉元素一多，单文件会变得很大，后续难改难维护，效果不可控，也很难稳定地按要求产出；更麻烦的是，原本的文字容易失去文章感，看起来像一个网页应用。<sup>4</sup>

这篇文章要把这套流程讲清楚：输入先收成一份事实来源，写作在受约束的组件里进行，关键位置停下来由你拍板，最后再验收和交付。看懂之后，你能判断哪些环节可以让 Agent 自己跑，哪些决定必须自己下，以及出了问题该修哪一块。<sup>5</sup>

## 2 真正可复用的，是这套工作方法

这套流程要验证的判断是：好的 Harness 可以迁移。上一期用一套框架让 Agent 全自动地制作视频，这一次把几乎同一套框架放到一件完全不同的事情上——把任何文字编辑成精美的文章。<sup>6</sup>

框架本身由六个部分组成：上下文管理、工具系统、执行编排、状态与记忆、反馈与观测、

约束与恢复。把做视频的流程和写文章的流程并排对照，会发现两者的骨架几乎一样。<sup>7</sup>

结论落在方法上：真正值得复用的，可能不是某一段提示词，也不是某一个组件，而是这套工作方法——把复杂任务拆成阶段，把关键决策变成检查点，把上下文写进文件，把输出收进协议，把质量问题交给独立审核，把修复限制在最小切片，再把审核与修复的细节沉淀回流程。<sup>8</sup>

它解决的是一个稳定性问题：确保 Agent 从能做出一次效果，变成能稳定生产一类效果。<sup>9</sup>

作者对这套方法的边界给得很宽：做 Harness 不一定要从头打造 Agent，把某一个垂直任务用 Skill 做稳，本身就是在做 Harness。适用范围也一样——换到周报、播客、课程讲义、技术文档、产品发布这些场景，只要任务足够复杂、需要状态、流程、检查点和验收标准，这套骨架就有价值。<sup>10</sup>

### 3 把“让模型写 HTML”换成“让模型组合组件”

---

核心的动作是换掉模型要写的东西。Agent 编写的并不是原始 HTML，而是一套专门为文章定义的底层组件协议：协议给出一组受约束的语义化组件，基本平替 Markdown 的全部语法；AI 只负责组合这些组件，结构和排版由库保证，不需要自己去想版式结构、样式一致性和前后一致性。<sup>11</sup>

作者给这套协议取名 Reacticle，是 React 加 article 的合成词。它的定位可以压成一句话：Markdown 让人能轻松地写文章，Reacticle 让 AI 能可控地生成长文 HTML。<sup>12</sup>



**图 2** 画面把组件库挡掉的问题列成一张清单：顶部写着 LIBRARY GUARANTEE 和“不需要 AI 去想这些”，中间是 AI 不用再纠结的四件事——div 还是 section、h2 还是 h3、间距设多少、前后一致性，右侧盖着“库已保证”的章。版式与样式的一致性因此由库兜住，而不是每次留给模型重新决定。<sup>13</sup>

只有固定组件会显得死板，所以协议留了一个叫 Raw 的逃生舱组件：任意 HTML 语法都能塞进去，时序图、交互式画块都能做。它带一条硬约束——Raw 里的所有样式必须遵循当前主题的风格。这样既给模型留了自由，又把各个主题的效果稳定性约束住。<sup>14</sup>

这样的主题目前有 11 套。<sup>15</sup>

每套主题都由两份东西组成：一份样式约束，定义颜色、字体、间距、阴影这些视觉变量；一份写给 AI 的说明，规定这套主题该配什么图、能不能用渐变和阴影、图表是不是要极简、能不能撞色描边。<sup>16</sup>

两层的关系可以用乐高讲清楚：Reacticle 是积木，规定有哪些零件、每种长什么样、颜色怎么搭配，管的是拼出来稳不稳、好不好看；beautiful-article 这个 Skill 是拼装说明书，规定先拼哪块、拼完检查什么、最后怎么交付，管的是生产过程可不可控。一个管输出质量，一个管生产流程，两层配合才能稳定出货。<sup>17</sup>

## 4 整套流程实际跑起来，是九个步骤和三个检查点

---

流程从整理素材开始，一共九个步骤；第三步、第五步会停下来等你确认，第九步之后还有最后一个检查点决定交付形态。<sup>18</sup>

开始之前有一段环境准备：选一个支持 Skill 的执行 Agent，示例用的是 Claude Code，Cursor、Codex 这类同样可以；在终端装好后用 `claude` 命令验证能正常启动；再配一个能力和上下文都够强的模型，作者这次换用的是 MiniMax M3；最后把 Skill 目录放进 `.claude/skills`，输入 `/beautiful-article` 能调起来，就说明装好了。<sup>19</sup>

第一步是整理素材。输入无论是网页链接、PDF 还是纯文本，都先被统一成一份 Markdown，也就是流程里的 `sources.md`，作为后面所有写作的事实来源；整理过程中拿不准的地方——图片缺失、表格不完整、链接可能失效——单独记录下来，交给后面的人工判断。<sup>20</sup>

第二步是出编辑方案。整理好的素材会变成一份完整的编辑计划（`plan.md`），写清楚面向谁、保留多少原文信息、章节怎么分、选什么视觉主题；这些都给建议，但不替你做决定。<sup>21</sup>

第三步是第一个检查点，流程停下来问你几件事：文章类型、主题风格、要不要目录、配图方式、封面方向。<sup>22</sup>

| checkpoint 1 • must confirm 它会问你 5 件事 |                                                                                                                |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 01 文章类型                               | <input checked="" type="radio"/> longform <input type="radio"/> report <input type="radio"/> tutorial          |
| 02 主题风格                               | <input checked="" type="checkbox"/> <input type="checkbox"/> <input type="checkbox"/> <input type="checkbox"/> |
| 03 要不要目录                              | <input checked="" type="checkbox"/> 显示目录                                                                       |
| 04 配图方式                               | <input checked="" type="radio"/> 真实图 <input type="radio"/> 占位 <input type="radio"/> AI生成                       |
| 05 封面方向                               | <input checked="" type="checkbox"/> <input type="checkbox"/> <input type="checkbox"/>                          |

询问刚刚帮你推荐好的这些关键事项

图 3 第一个检查点停在动手写正文之前：画面把要确认的五项排成一张表——文章类型（longform、report、tutorial）、主题风格、要不要目录、配图方式、封面方向，左上角标着 checkpoint 1 • must confirm，页脚写着“询问刚刚帮你推荐好的这些关键事项”。<sup>23</sup>

检查点的分寸被说得很清楚：Agent 会根据原文类型推荐最合适的选项，但不会替你决定。<sup>24</sup>

文章类型这一项直接决定文章以什么形态呈现。内置类型涵盖完整长文、研究报告、教学步骤、机制解释、对话访谈、工程审阅、观点评论、角色摘要和交互式学习等，这次示范选的是 100% 保留原文内容的完整长文。<sup>25</sup>

第四步不是一口气写完，而是先出一小块：只做封面、第一节和一个有代表性的内容模块，用来验证正文密度合不合适、主题对不对；验证过了再继续往下写。<sup>26</sup>

第五步是第二个检查点，这次请你对首屏提意见：标题是不是太正经、配色喜不喜欢、信息是不是太密，都可以在这一步调整。这里改写的成本最低；如果拖到后面，写得越多，返工的成本就越高。<sup>27</sup>

第六步是定开发模式：一次性顺序写，还是多 Agent 并行写。顺序模式风格最统一，并行模式速度最快；作者更推荐并行——更快，也在一定程度上更省 token，而且 Skill 的文件约束保证了并行不会导致冲突。<sup>28</sup>

第七步是逐章开发，确认首屏之后才进入这一步。每一节写成独立文件，每个模块也是独立组件，多个 Agent 并行时各写各的，最后主 Agent 统一合并；这条文件边界既是能并行的前提，

也是后面能做最小切片修复的前提。<sup>29</sup>

第八步是终审：所有章节写完，流程还不会结束，三个独立的审查者各审一路，维度各不相同。<sup>30</sup>

内容审查看完整性、信息取舍和结构；视觉审查看主题、Raw、配图和移动端适配是否合理；技术审查看构件和空态有没有报错、代码块与公式渲染是否正常。<sup>31</sup>

关键的质检节点必须交给独立的审查者，不能由写作的那一路自己点头。<sup>32</sup>

第九步是最小切片修复：审查发现的问题不做整体重写，某一节有问题就改哪一节，某个Raw有问题就改哪个Raw；同时对已经确认过的部分加以保护，避免一次局部返工把整篇文章重新搞坏。<sup>33</sup>

最后一个检查点决定交付形态：只导出一个HTML，还是同时导出HTML加PDF，又或者先把还没处理的问题修掉。演示里选的是同时导出HTML和PDF，网页和PDF的内容完全一致，但排版会按各自的格式做适配。<sup>34</sup>

整套流程里，关键状态都被写进文件：sources.md记录抽取结果和风险，plan.md记录所有编辑决策与大纲，质检和修复也各有记录。这些文件合起来构成Agent的工作记忆，写到后面章节时不用凭记忆回忆，直接回去读文件即可。<sup>35</sup>



图4 画面标题是「所有关键点的痕迹都落到本地」：一个review/目录把质检和审查两类记录沉淀下来，底部字幕写着「就是所有的关键质检过程」。<sup>36</sup>



## 5 把它压成一张可以逐条自查的清单

---

这套方法的价值不在于你用了哪个模型或哪个组件，而在于几个固定动作。下面这张清单综合了整条流程里散落的几条原则，把它压成问题，就可以直接拿来审自己的流程。<sup>37</sup>

- 关键决策有没有停下来让你拍板？检查点只给建议，不替你决定，而且停在改写成本最低的位置。
- 上下文有没有分阶段加载？初始就把所有规范、主题和组件文档塞进去会稀释注意力，不同阶段只该看到这个阶段需要的文档。
- 每写一节之前，有没有回看当前任务的协议、Raw 规范和主题约束？这是把流程持续拉回正轨的做法。
- 输入有没有统一成一份事实来源？链接、PDF、纯文本都先整理成同一份文件，后面所有写作都从它出发。
- 每一节是不是独立的文件？并行开发需要各写各的，最小切片修复也需要文件边界。
- 状态有没有写进文件？计划、抽取结果、风险和质检记录落到本地，写到后面章节时直接回去读文件。
- 质检有没有独立视角？内容、视觉、技术三路各审各的，关键节点必须由独立审查者完成。
- 修复有没有限制在最小切片？只改出问题的那一节或那一个组件，并保护已经确认过的部分。<sup>38</sup>

还有一件可以立刻做的事：把关键的中间过程落到本地文件——首屏审查、终审和修复记录都算。这些文件不只给人看，也给 Agent 看；同类任务跑多之后，Agent 回看这些记录，能找出最容易出问题的环节，比如目录写错、Raw 过度设计、某类文章的信息保留比例不合适，再反过来优化 Skill 的规则、检查清单和默认策略。<sup>39</sup>

把这些动作固定下来，最后留下来的除了一个 HTML 文件，还有一条被验证过的生产流程。<sup>40</sup>

## 来源时间

正文中的上标编号对应下面列出的时间范围，标注该段内容在视频中的位置。

- |                                                                   |                                                                   |
|-------------------------------------------------------------------|-------------------------------------------------------------------|
| 1. 00:00:00--00:00:10, 00:01:00--00:01:06                         | 22. 00:02:52--00:03:01                                            |
| 2. 00:01:12--00:01:31                                             | 23. 00:02:58--00:03:02                                            |
| 3. 00:00:58--00:01:02                                             | 24. 00:10:35--00:10:40                                            |
| 4. 00:01:34--00:01:57                                             | 25. 00:10:40--00:11:36, 00:11:48--00:11:49                        |
| 5. 00:18:09--00:18:11, 00:19:37--00:19:49                         | 26. 00:03:03--00:03:16                                            |
| 6. 00:00:12--00:00:16, 00:00:37--00:00:50                         | 27. 00:03:16--00:03:24                                            |
| 7. 00:15:47--00:15:52, 00:15:56--00:16:08                         | 28. 00:03:24--00:03:37, 00:13:23--00:13:36                        |
| 8. 00:19:29--00:19:49                                             | 29. 00:13:48--00:14:10, 00:17:01--00:17:17                        |
| 9. 00:19:49--00:19:56                                             | 30. 00:03:40--00:03:45, 00:14:15--00:14:18                        |
| 10. 00:19:56--00:20:15                                            | 31. 00:03:45--00:03:56, 00:14:21--00:14:39                        |
| 11. 00:04:09--00:04:18, 00:04:44--00:04:56                        | 32. 00:18:06--00:18:11                                            |
| 12. 00:04:18--00:04:31                                            | 33. 00:03:56--00:03:58, 00:14:39--00:14:50,<br>00:18:33--00:18:46 |
| 13. 00:04:58--00:05:02                                            | 34. 00:03:58--00:04:07, 00:14:50--00:15:05,<br>00:15:19--00:15:24 |
| 14. 00:04:57--00:05:19                                            | 35. 00:10:04--00:10:28, 00:17:17--00:17:44                        |
| 15. 00:05:19--00:05:22                                            | 36. 00:18:58--00:19:02                                            |
| 16. 00:05:22--00:05:47                                            | 37. 00:15:56--00:15:58, 00:19:29--00:19:35                        |
| 17. 00:05:58--00:06:26                                            | 38. 00:16:11--00:20:15                                            |
| 18. 00:02:26--00:04:07                                            | 39. 00:18:55--00:19:27                                            |
| 19. 00:06:36--00:07:02, 00:07:33--00:07:36,<br>00:08:40--00:09:09 | 40. 00:20:17--00:20:19                                            |
| 20. 00:02:26--00:02:40, 00:16:53--00:17:01                        |                                                                   |
| 21. 00:02:40--00:02:52                                            |                                                                   |